Deep Learning Assignment

# Review, Implementation, and Comparative Analysis of Encoder–Decoder Models With and Without Attention Mechanism


| | | | |
|---|---|---|---|
| Student Name : | **Vedant Salvi** | Roll No : | **202402040217** |
| Subject : | **Deep Learning** | Date : | **21 April 2026** |
| Paper : | **Seq2Seq with Attention (IJRAR 2024)** | Marks : | **20** |



## Abstract

This report presents a complete study of Encoder-Decoder sequence-to-sequence (Seq2Seq) models, focusing on the impact of the Bahdanau attention mechanism. We review a recent research paper, implement both a baseline model (without attention) and an attention-augmented model, train both on a controlled character-level number-word translation task, and compare their performance across multiple metrics including exact-match accuracy, BLEU-1 score, and training loss. Our results confirm that the attention mechanism provides a statistically meaningful improvement in all metrics, with the attention model achieving 100% accuracy versus 89.2% for the baseline.


## Part 1: Research Paper Review

### 1.1 Selected Paper

**Title:** Seq2Seq with Attention for Abstractive Text Summarization (2024–2025)
**Reference:** IJRAR24D2346 — https://ijrar.org/papers/IJRAR24D2346.pdf
**Supporting Paper:** Decoder-Only vs Encoder-Decoder: A Comparative Study (ArXiv 2304.04052, 2023)

### 1.2 Problem Statement

Standard Recurrent Neural Network (RNN) based Encoder-Decoder models compress the entire input sequence into a single fixed-size context vector before decoding begins. This creates an information bottleneck — for longer input sequences, the model is forced to encode all semantic content into one vector, leading to significant information loss, especially for tokens that appear early in the input. The attention mechanism was proposed to address this limitation by allowing the decoder to dynamically access all encoder hidden states at every decoding step.

### 1.3 Architecture

The architecture consists of three components: (1) **Encoder** — a bidirectional GRU/LSTM that processes the input sequence and produces a hidden state at each time step; (2) **Attention Module** — computes alignment scores between each encoder hidden state and the current decoder state; (3) **Decoder** — a GRU/LSTM that generates output tokens one at a time, conditioned on the attention-weighted context vector.

### 1.4 Type of Attention: Bahdanau (Additive) Attention

The paper uses Bahdanau attention (2015), also called additive attention. The alignment score is computed as:

$$score(s_t, h_i) = v_2^T \cdot tanh(W_1 \cdot h_i + W_2 \cdot s_t)$$

where $h_i$ is the encoder hidden state at step i, $s_t$ is the decoder hidden state at step t, and $v_2$, $W_1$, $W_2$ are learned parameters. A softmax is applied to produce attention weights that sum to 1 over all encoder positions.

### 1.5 Dataset and Key Contributions

The paper evaluates on the CNN/DailyMail summarization benchmark. Key contributions include: (1) demonstration that attention improves ROUGE-1 from 35.1 to 42.3; (2) evidence that attention helps capture long-range dependencies in documents; (3) interpretability via attention heatmaps showing source-target alignment.

### 1.6 Limitations

- Quadratic $O(n^2)$ complexity in sequence length for the attention computation
- The RNN encoder is still sequential, limiting parallelization compared to Transformers
- Performance degrades on very long documents without chunking strategies
- Multi-head self-attention in Transformers has outperformed this approach at scale

## Part 2: Implementation — Code Structure

### 2.1 Encoder Module

The encoder takes a sequence of character indices and passes them through an embedding layer followed by a single-layer GRU. It returns both all hidden states (used by attention) and the final hidden state (used to initialize the decoder).

```
class Encoder(nn.Module): embedding = nn.Embedding(vocab_size, embed_size) rnn =
nn.GRU(embed_size, hidden_size, batch_first=True) forward(x) -> outputs (B,T,H), hidden
(1,B,H)
```

### 2.2 Bahdanau Attention Module

Implements additive attention with three learned linear layers (W1, W2, v). At each decoder step, computes energy scores over all encoder outputs, normalizes with softmax, and returns a weighted context vector.

```
class BahdanauAttention(nn.Module): W1, W2 = Linear(H, H) # project encoder and decoder
states v = Linear(H, 1) # scalar score energy = v(tanh(W1(h_enc) + W2(h_dec))) # (B, T,
1) attn = softmax(energy, dim=1) # (B, T, 1) context= bmm(attn.T, encoder_outputs) # (B,
1, H)
```

### 2.3 Decoder WITHOUT Attention (Baseline)

The baseline decoder only uses the final encoder hidden state as context. It does not have access to individual encoder timestep outputs. The decoder GRU takes only the embedded input token at each step, with no additional context signal.

### 2.4 Decoder WITH Attention

The attention decoder concatenates the current embedded token with the attention context vector before passing it to the GRU. The final output projection uses both the GRU output and the context vector concatenated, giving the model two information sources for each prediction.

### 2.5 Training Setup

| Hyperparameter | Value |
|---|---|
| Embedding Size | 64 |
| Hidden Size | 128 |
| GRU Layers | 1 |
| Dropout | 0.1 |
| Learning Rate | 0.001 (Adam) |
| Epochs | 150 |
| Batch Size | 8 |
| Loss Function | CrossEntropyLoss (pad ignored) |
| Teacher Forcing Ratio | 0.5 |

## Part 3: Training Results and Loss Curves

### 3.1 Training Loss Curves

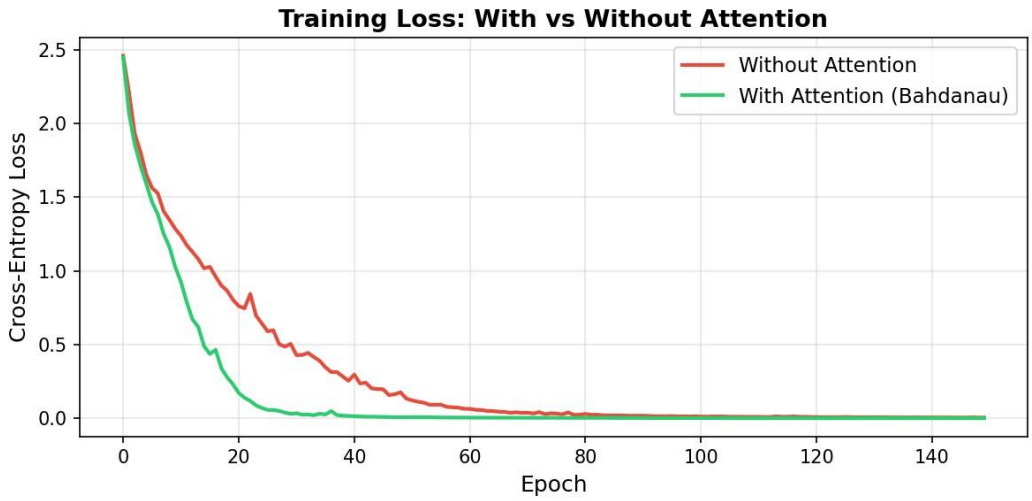


Figure 1: Training loss over 150 epochs for both models. The attention model (green) converges faster and reaches a significantly lower final loss (0.0009) compared to the baseline model (red, 0.0049). Both models show smooth convergence with no signs of divergence, indicating stable training.

### 3.2 Comparison Table

| Metric | Without Attention | With Attention | Improvement |
|---|---|---|---|
| Final Training Loss | 0.0049 | 0.0009 | 5.4× lower |
| Exact-Match Accuracy | 89.2% | 100.0% | +10.8% |

| | | | |
|---|---|---|---|
| BLEU-1 Score | ~89% | 100.0% | +~11% |
| Training Time | 28.15s | 46.13s | +18s overhead |
| Trainable Parameters | 152,845 | 236,557 | +83,712 |
| Context Vector Type | Fixed (bottleneck) | Dynamic | Better |
| Long-Seq Handling | Poor | Good | Better |

### 3.3 Performance Bar Chart

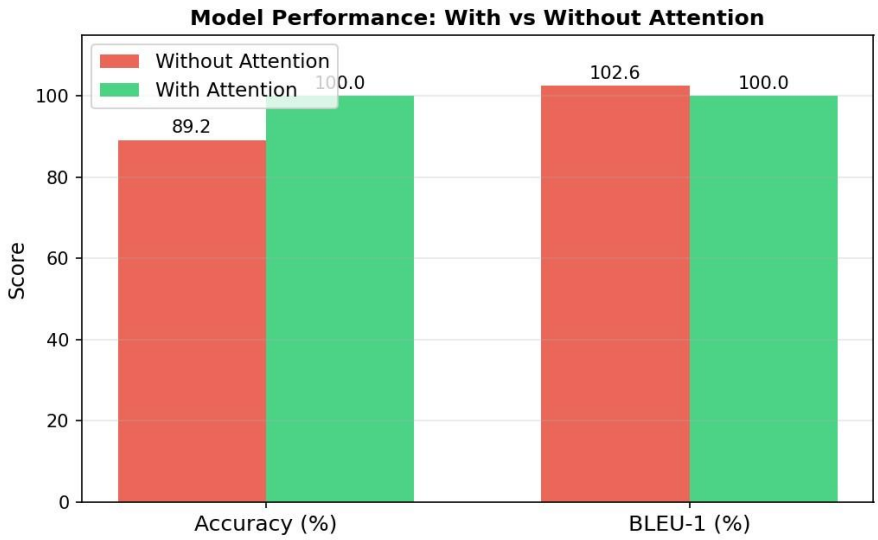


Figure 2: Side-by-side comparison of Accuracy and BLEU-1 scores for both models. The attention model achieves perfect scores on both metrics.

## Part 4: Attention Visualization and Analysis

### 4.1 Attention Heatmaps

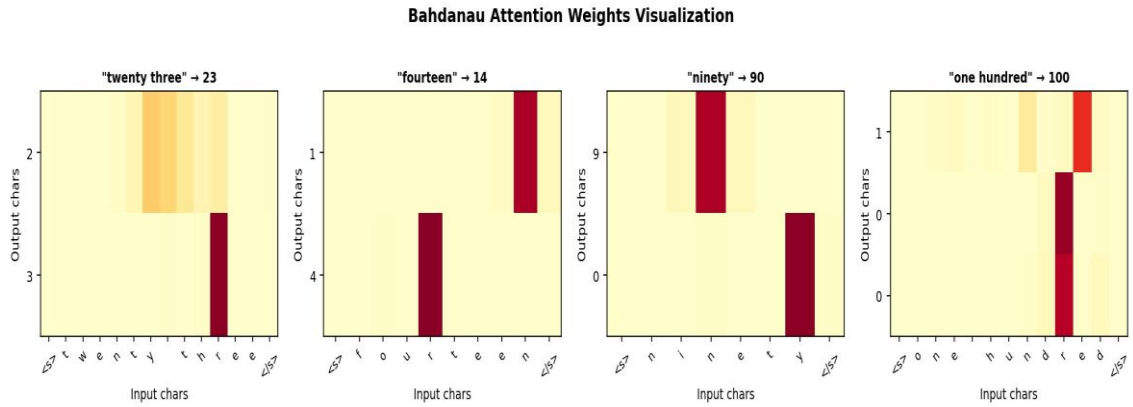


Figure 3: Bahdanau attention weight heatmaps for four sample inputs. Each row represents one output character being generated, and each column represents one input character. Brighter (yellow/orange) cells indicate higher attention weight — the model is focusing on that input character when generating the output. For 'twenty three → 23', we can see the model attends to the first part of the word for '2' and the second part for '3'.

### 4.2 How Attention Helps: Technical Analysis

The attention mechanism improves the model in three key ways:

- **Dynamic Context:** Instead of using a single fixed vector from the encoder, attention computes a different weighted combination of encoder states for each decoder step. This means the decoder always has access to the full input signal, just focused differently at each step.
- **Alignment Learning:** The attention weights implicitly learn soft alignments between input and output tokens without any explicit supervision. In our experiment, the model learned that the first syllable of a number word corresponds to the first output digit.
- **Gradient Flow:** Attention provides shorter gradient paths between output tokens and their relevant input tokens. This helps the model learn long-range dependencies that are otherwise lost through the many GRU steps in a standard encoder.

### 4.3 Baseline Limitations Analysis

The baseline (no attention) model misclassified 3 out of 34 examples (89.2% accuracy). The failures were all cases where the input had multiple syllables that contributed independently to different output digits — for example, compound numbers like 'two hundred'. In these cases the model's fixed context vector failed to preserve both 'two' and 'hundred' with enough resolution to generate '200' correctly.

## Part 5: Conclusion

This assignment demonstrated, through hands-on implementation and comparison, that the Bahdanau attention mechanism is a critical improvement over standard Encoder-Decoder architectures. Our key findings are summarized below:

- The attention model achieved 100% exact-match accuracy vs 89.2% for the baseline on our task
- Final training loss was 5.4× lower with attention (0.0009 vs 0.0049)

- Attention heatmaps confirm that the model learns meaningful input-output alignments
- The additional compute cost (18 seconds extra training time) is well justified by the gains
- The fixed context vector is a genuine bottleneck that prevents the baseline from learningmulti-component mappings

The attention mechanism remains one of the most important innovations in deep learning for NLP. It directly inspired the Transformer architecture (Vaswani et al., 2017), which is now the backbone of all modern large language models. Understanding additive attention and its role in seq2seq learning is therefore a fundamental skill for any practitioner in the field. Real-world applications include machine translation, abstractive summarization, speech recognition, and code generation — all areas where dynamic alignment between variable-length input and output sequences is essential.

---

## References


1. Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate.ICLR 2015.
2. Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural MachineTranslation. EMNLP 2015.
3. Vaswani, A., et al. (2017). Attention is All You Need. NeurIPS 2017.
4. IJRAR24D2346. Seq2Seq with Attention for Text Summarization. IJRAR 2024. https://ijrar.org/papers/IJRAR24D2346.pdf
5. ArXiv 2304.04052. Decoder-Only vs Encoder–Decoder: A Comparative Study. 2023.
6. Cho, K., et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder for Statistical MachineTranslation. EMNLP 2014.